

UNIT - VI

6

Computability and Complexity Theory

Syllabus

Computability Theory : Decidable Problems and Un-decidable Problems, Church-Turing Thesis.

Reducibility : Undecidable Problems that is recursively enumerable, A Simple Un-decidable problem.

Complexity Classes : Time and Space Measures, The Class P, Examples of problems in P, The Class NP, Examples of problems in NP, P Problem Versus NP Problem, NP-completeness and NP-hard Problems.

Contents

Part I : Computability Theory

6.1 Decidable Problems and Un-decidable Problems

..... **May-06,10,15,16,**
..... **Dec.-07,11,12,13** Marks 12

6.2 Church-Turing Thesis

Part II : Reducibility

6.3 Undecidable Problems that is Recursively Enumerable

..... **Dec.-05,07,08,16,17,**
..... **May-08,09,10,18,19,** Marks 8

6.4 A Simple Un-decidable Problem. **May-07,12,14,18,**

..... **Dec.-10,11,12,19,** Marks 8

Part III : Complexity Classes

6.5 Time and Space Measures

6.6 The Class P and NP **Dec.-15,18,19,** Marks 4

6.7 Examples of Problems in P **Dec.-17,18,** Marks 8

6.8 Examples of Problems in NP **May-16,18, Dec.-17,19,** Marks 8

6.9 NP-completeness and NP-hard Problems **Dec.-16,** Marks 8

6.10 Polynomial Time Reduction **May-08,16, Dec.-18,19,** Marks 8

6.11 Satisfiability Problems **Dec.-16,18,** Marks 8

6.12 Restricted Satisfiability Problem

Part I : Computability Theory

6.1 Decidable Problems and Un-decidable Problems

SPPU : May-06,10,15,16, Dec.-07,11,12,13, Marks 12

- If a language is recursive then it is called decidable language and if the language is not recursive then such a language is called undecidable language.
- The class of decidable problems are also called as solvable problems and the class of undecidable problems is called unsolvable problems.

6.1.1 Decidability Problems Concerning Regular Languages

SPPU : May-15, 16

The decidability problems concerning regular languages can be understood with the help of following theorems.

Theorem 1 : Prove that following decision problems are recursive

(i) Two DFA's are equivalent or Not (ii) NFA Accepts a word or not

SPPU : May-15 End Sem, Marks 10, May-16, End Sem, Marks 5

Proof : (i)

- To prove that the given decision problem is recursive. We require a TM $T(M)$ which simulates the DFA.
- Let, M_1 and M_2 are two DFAs. Consider the Turing machines $T(M_1)$ and $T(M_2)$. Obtain set of strings accepted by $T(M_1)$ and rejected by $T(M_2)$. That is

$$S_1 = T(M_1) \cap \overline{T(M_2)}.$$

- Similarly obtain the set of strings which $T(M_2)$ accepts and $T(M_1)$ rejects. That is

$$S_2 = T(M_2) \cap \overline{T(M_1)}.$$

- If these two sets are empty then it shows that $L(M_1)$ and $L(M_2)$ are equivalent and can be simulated by TM for the set of languages as.

$$L(M) = L(M_1) \cap \overline{L(M_2)} \cup \overline{L(M_1)} \cap L(M_2). \text{ The } L(M) = \phi.$$

- This shows that two DFA's are equivalent or not is a recursive problem.

(ii) Let, M be the NFA and input word w which decides if M accepts w . Convert this NFA to DFA. Then run the algorithm for DFA on Turing Machine $T(M)$. The TM $T(M)$ accepts w if and only if M reaches a final state on w . This shows that NFA accepts a word or not is recursive.

Theorem 2 : Prove that following decision problems is recursive : DFA Accepts a word or not.

SPPU : May-15, End Sem, Marks 5

Proof :

Let, M be the DFA and input word w which decides if M accepts w . Simulate an algorithm for DFA to run on Turing machine $T(M)$. The TM $T(M)$ accepts w if and only if M reaches a final state for w otherwise rejects. This show that DFA accepts a word or not is recursive.

6.1.2 Decidability Problems Concerning Context Free Languages

SPPU : May-06,10,15,16, Dec.-07,11,12,13

The undecidable problems about CFG are -

- 1) Whether or not two different context free languages define the same language ?
- 2) Whether given CFL is ambiguous or not ?
- 3) Whether complement of given CFL is context free language ?
- 4) Whether the intersection of two context free languages is context free ?

There is no algorithm to answer these questions. Hence these problems are known as **undecidable** problems.

Theorem 3 : Show that the following problem is undecidable. "Given two CFGs G_1 and G_2 , is $L(G_1) \cap L(G_2) = \phi$ ".

Proof :

Let $L(G_1)$ and $L(G_2)$ be two CFLs. The CFL is not closed under intersection. That means $L(G_1) \cap L(G_2)$ does not necessarily generates CFL. This also implies that there is no algorithm which can tell whether $L(G_1) \cap L(G_2) = \phi$. Hence it is said that $(L(G_1) \cap L(G_2) = \phi)$ is undecidable.

Theorem 4 : Prove that CFG G generates the string w or not.

SPPU : May-15, End Sem, Marks 5, May-16, End Sem, Marks 5

Proof :

Convert CFG G to G'' in chomsky normal form. Now the string $w \in L(G'')$ iff w can be derived in $2|w|-1$ steps where none of the intermediate string is of length more than $|w|$. If these derived step will derive w , the $T(M)$ accepting $L(G'')$ will accept otherwise rejects. This shows CFG G generates string w or not is recursive.

Theorem 5 : Let G be a CFG and r be a regular expression. Show that the problem
 1) $L(G) = L(r)$ 2) $L(r) \in L(G)$ are undecidable.

SPPU : May-06, 10, Dec.-07, Marks 12, Dec.-11, Marks 8

Proof :

- 1) Let, $L(G)$ is a context free language and $L(r)$ represents a regular language having regular expression. The post's correspondence problem PCP has some instance which becomes a solution for given language $L(r)$ can be accepted by the CFG for $L(G)$. It is also possible that there is no solution instance of PCP. Such an instance is not accepted by CFG. Thus there may exist a CFG for $L(r)$ or there may not be a CFG for $L(r)$. Hence we can state that $L(G) = L(r)$ is undecidable.
- 2) Let $L(r)$ represents a regular language for some regular expression r . The $L(G)$ denotes the context free language for a CFG G . There may be some strings of language $L(r)$ that are acceptable by G . Then we can state $L(r) \in L(G)$.

But since these are arbitrary strings, there may be some strings of $L(r)$ which are not acceptable by $L(G)$. Hence $L(r) \notin L(G)$.

Thus it is not possible to determine whether $L(r) \in L(G)$. Hence it is undecidable

Theorem 6 : It is undecidable whether a CFG is ambiguous.

SPPU : Dec.-13, Marks 6

Proof :

Let $A = (x_1, x_2, \dots, x_k)$ and $B = (y_1, y_2, y_3, \dots, y_k)$ are two strings.

The G_1 and G_2 are two context free grammars. The production rules for these grammars are as follows -

$$S_1 \rightarrow a_i S_1 x_i \quad \text{for } i = 1, 2, 3, \dots, k$$

$$S_1 \rightarrow a_i x_i$$

$$S_2 \rightarrow a_i S_2 y_i \quad \text{for } i = 1, 2, 3, \dots, k$$

$$S_2 \rightarrow a_i y_i$$

The grammar G_1 generates the sentence of the form

$$a_{i_n} a_{i_{n-1}} \dots a_{i_1} x_{i_1} x_{i_2} \dots x_{i_n}$$

The grammar G_2 generates the sentence of the form

$$a_{i_n} a_{i_{n-1}} \dots a_{i_1} y_{i_1} y_{i_2} \dots y_{i_n}$$

We can combine the languages from both the grammar. The new production which is added here is -

$$S \rightarrow S_1 \mid S_2$$

If grammar G_{12} is ambiguous then PCP with pair (A, B) has a solution.

Similarly if G_{12} is unambiguous then PCP with pair (A, B) has no solution.

But since having solution to PCP is undecidable and there is algorithm for decidability of PCP, there is no algorithm deciding whether given CFG is ambiguous or not. Hence whether CFG is ambiguous or not is undecidable.

Review Question

1. Explain with example decidable and non decidable language.

SPPU : May-16, End Sem, Marks 6

6.2 Church-Turing Thesis

Hypothesis means proposing certain facts. The Church's hypothesis or Church's Turing thesis can be stated as,

"The assumption that the intuitive notion of computable functions can be identified with partial recursive functions".

However, this hypothesis cannot be proved. The computability of recursive functions is based on following assumptions -

- 1) Each elementary function is computable.
- 2) Let f be the computable function and g be the another function which can be obtained by applying an elementary operation to f , then g becomes a computable function.
- 3) Any function becomes computable if it is obtained by rule 1) and 2).

Part II : Reducibility

6.3 Undecidable Problems that is Recursively Enumerable

SPPU : Dec.-05,07,08,16,17, May-08,09,10,18,19, Marks 8

The recursively enumerable (RE) languages are categorized into two classes -

1. The class of languages that has Turing machine. This Turing machine decides whether the input string belongs to that language or not. Such a Turing machine always halts, whether or not it reaches to accept state.

2. The second class of languages consists of those RE languages that are not accepted by any Turing machine with the guarantee of halting.

A language is L actually denoted by $L(M)$ called **recursive** if it is accepted by some Turing machine such that

1. If string w is in L , turing machine M accepts it and then halts.
2. If w is not in L , then M eventually halts although it never enters in accepting states.

We can also call recursive languages as the **definite languages** or the languages which can be represented by some **algorithm** and such an algorithm helps in construction of Turing machine for that language.

Decidable and Undecidable Languages -

If a language is **recursive** then it is called **decidable languages** and if the language is **not recursive** then such a language is called **undecidable** language.

Hence broadly there are three categories of the languages.

1. Recursive language for which the algorithm exists.
2. Recursively enumerable language for which it is not sure that on which input the TM will ever halt. Such languages are not recursive.
3. The non-recursively enumerable languages for which there is no Turing machine at all.

Fig. 6.3.1 shows the relationship between these languages.

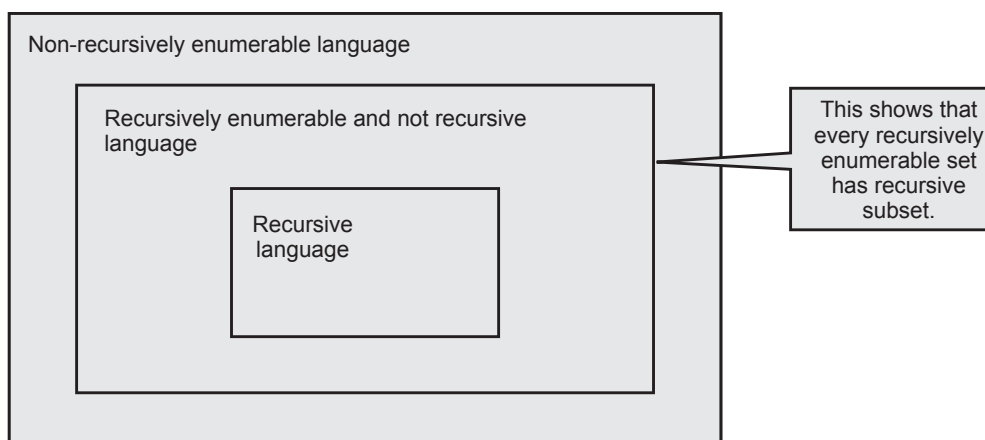


Fig. 6.3.1 Relationship between languages

The important fact about recursive and recursively enumerable languages can be seen with the help of following theorem.

Theorem 1 : If L is recursive language then L' is also a recursive language.

Proof : Let there will be some L that can be accepted by turing machine M . Hence we can denote language L by $L(M)$. On acceptance of $L(M)$ the machine M always halts. Now, we construct a TM M' such that $L' = (M')$. For construction of M' following steps are followed -

1. The accepting steps of M are made non-accepting states of M' and there is no transition from M' . That means we have created the states such that M' will halt without accepting.
2. Now create a new accepting state for M' say r and there is no transition from r .
3. In machine M , for each of the transition with combination of nonaccepting state and input tape symbol, make the same transition having the combination of accepting state and input tape symbol for machine M' .

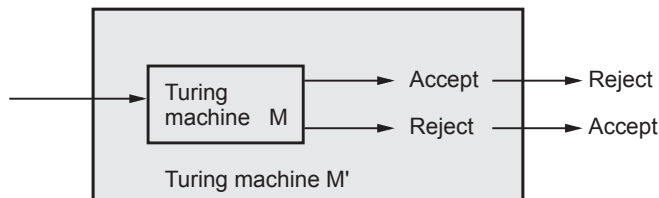


Fig. 6.3.2 Construction of M' accepting L'

Since M is guaranteed to halt M' is also guaranteed to halt. In fact, M' accepts exactly those strings that M does not accept. Thus we can say that M' accepts L' .

Theorem 2 : If a language L and its complement L' both are RE then L is a recursive language.

SPPU : Dec.-05, Marks 6

Proof : Consider a turing machine M made up of two turing machines M_1 and M_2 . The machine M_2 is complement of machine M_1 . We can also denote that $L(M) = L(M_1)$

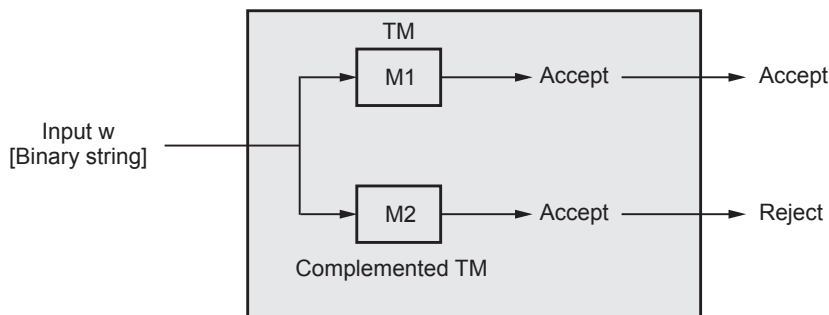


Fig. 6.3.3

and $L(M_2)$. Both M_1 and M_2 are simulated in parallel by machine M . Machine M is a two tape TM, which can be made one tape TM for the ease of simulation. This One tape then will consist of tape of machine M_1 and machine M_2 . The states of M consists of all the states of machine M_1 and all the states of machine M_2 . The machine M made up of M_1 and M_2 is as shown in Fig. 6.3.3.

If the input w of language L is given to M then M_1 will eventually accept and therefore M will accept L and halt. If w is not in L , then it is in L' . So M_2 will accept and therefore M will halt without accepting. Thus on all inputs, M halts. Thus $L(M)$ is exactly L . Since M always halts we can conclude that $L(M)$ mean L is a recursive language. Thus a recursive language can be recursively enumerable but a recursively enumerable language is not necessarily be recursive.

Theorem 3 : Show that if L_1 and L_2 are recursive languages then $L_1 \cup L_2$ and $L_1 \cap L_2$ also recursive.

SPPU : May-08, 10, Marks 8, Dec.-16, End Sem, Marks 6

Proof : Let

L_1 is a recursive language.

L_2 is a recursive language.

As L_1 and L_2 are recursive languages there exists a machine M_1 that accepts L_1 as well as machine M_2 that accepts L_2 . Now, we have to simulate a machine (algorithm) M that accepts the language L such that $L = L_1 \cup L_2$. Then construct machine M which accepts if M_1 accepts. The construction of M is as shown in following Fig. 6.3.4.

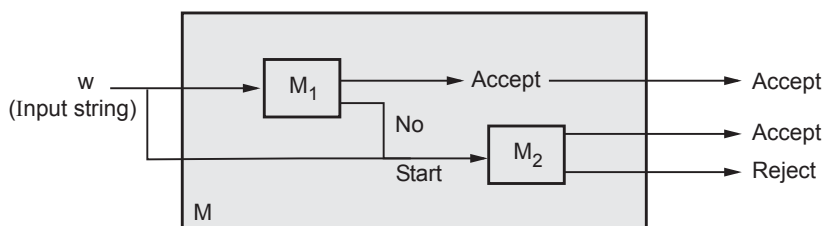
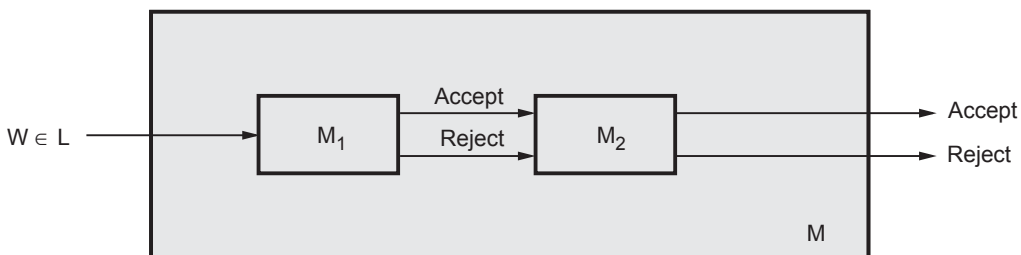


Fig. 6.3.4

If machine M_1 does not accept then M_2 simulates M . That means if M_2 accepts then M accepts, if M_2 rejects M also rejects. Thus M accepts the language $L = L_1 \cup L_2$ which is recursive.

To prove $L_1 \cap L_2$ as a recursive language consider machine M that accepts the language L such that $L = L_1 \cap L_2$.



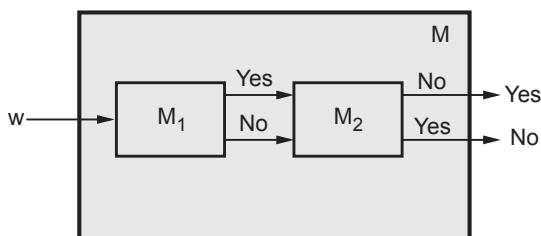
M_1 is a TM which accepts L_1 and M_2 is a TM which accepts L_2 . The TM M is simulated which halts on accepting $w \in L$ such that $L = L_1 \cap L_2$. Hence intersection of two recursive languages is also recursive.

Theorem 4 : If L_1 and L_2 are two recursive languages and if L is defined as :
 $L = \{w \mid w \text{ is in } L_1 \text{ and not in } L_2 \text{ and not in } L_1\}$. Prove or disprove
 that L is recursive.

SPPU : Dec.-08, Marks 8

Proof : Let L_1 is a recursive language which is accepted by M_1 and L_2 is a recursive language which is accepted by M_2 . Languages L_1 and L_2 are such that if $w \in L_1$ then $w \notin L_2$. Similarly if $w \notin L_1$ then $w \in L_2$.

Find $L = L_1 \cup L_2$. We can then design a TM M which accepts the language L . Such a TM can be drawn as follows -



As the language L is accepted by TM M , we can say that the language L is recursive language.

Theorem 5 : Show that the set of languages L over $\{0, 1\}$ so that neither L nor L' is recursively enumerable is uncountable.

SPPU : Dec.-07, May-09, 10, Marks 6

Proof : If there is any language L which is recursively enumerable then there exists a TM which semidecides it (either accepts and halt or loops for ever).

Every TM has description of finite length. Hence number of TM and number of recursively enumerable languages is countably. infinite, because power set of every countable set is countably infinite. Now, for the language L its complement L' is RE.

Then for L' also there exists TM which semidecides it. But there are uncountable number of languages. Hence there may be language L and L' which are not recursively enumerable. And there may be uncountable number of such languages. Hence neither L nor L' is RE in uncountable.

Theorem 6 : Prove that i) $AREX = \{ \langle R, W \rangle \mid R \text{ is a regular expression that generates string } w \}$ is a decidable language.
 ii) $ECFG = \{ \langle G \rangle \mid G \text{ is a CFG and } L(G) = \phi \}$ is a decidable language.

SPPU : May-19, End Sem, Marks 8

Proof :

i)

- Let M be the DFA and input word $w \in R$ which is accepted by M .
- Simulate an algorithm for DFA to run on Turing Machine TM
- The TM accepts w if and only if M reaches a final state for w otherwise rejects.
- This shows that the regular expression generates string that denotes decidable language.

ii)

- We construct TM M to decide a problem.
- Scan input string $\langle G \rangle$, determining whether the input constitutes a valid CFG. If not, then reject.
- Mark every terminal in every rule.
- For each rule $A \rightarrow w$, if every symbol of w is marked, mark every occurrence of A in rules.
- Continue until no new variables are marked.
- If S (Start symbol) is not marked then accept $\langle G \rangle$ otherwise reject.
- This shows that if G is a CFG then $L(G) = \phi$ is decidable language.

Review Questions

1. Define and explain recursive and recursively enumerable languages.

SPPU : Dec.-17, End Sem, Marks 8

2. Write short note - Undecidability.

SPPU : May-18, End Sem, Marks 4

6.4 A Simple Un-decidable Problem

SPPU : May-07,12,14,18, Dec.-10,11,12,19, Marks 8

6.4.1 Post Correspondence Problem

In this section, we will discuss the undecidability of strings and not of Turing machines. The undecidability of strings is determined with the help of Post's Correspondence Problem (PCP). Let us define the PCP.

"The post's correspondence problem consists of two lists of strings that are of equal length over the input Σ . The two lists are $A = w_1, w_2, w_3, \dots, w_n$ and $B = x_1, x_2, x_3, \dots, x_n$ then there exists a non empty set of integers $i_1, i_2, i_3, \dots, i_n$ such that, $w_{i_1} w_{i_2} w_{i_3} \dots w_{i_n} = x_{i_1} x_{i_2} x_{i_3} \dots x_{i_n}$ "

To solve the post correspondence problem we try all the combinations of $i_1, i_2, i_3, \dots, i_n$ to find the $w_i = x_i$ then we say that PCP has a solution.

Example 6.4.1 Consider the correspondence system as given below -

$A = (1; 0; 010; 11)$ and $B = (10; 10; 01; 1)$. The input set is $\Sigma = \{0, 1\}$. Find the solution.

Solution : A solution is 1; 2; 1; 3; 3; 4. That means $w_1 w_2 w_1 w_3 w_3 w_4 = x_1 x_2 x_1 x_3 x_3 x_4$.

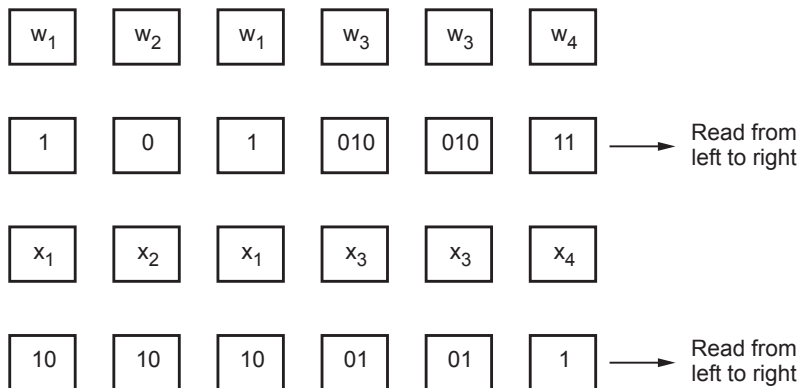


Fig. 6.4.1

The constructed string from both lists is 10101001011.

Example 6.4.2 Obtain the solution for the following correspondence system.

$A = \{ba, ab, a, baa, b\}$, $B = \{bab, baa, ba, a, aba\}$. The input set $\{a, b\}$.

Solution : To obtain the corresponding system the one sequence can be chosen. Hence we get $w_1 w_5 w_2 w_3 w_4 w_4 w_3 w_4 = x_1 x_5 x_2 x_3 x_4 x_4 w_3 w_4$. This solution gives a unique string babababaabaaba = babababaabaaba. Hence solution is 15234434.

Example 6.4.3 Obtain the solution for the following system of posts correspondence problem.

$$A = \{100, 0, 1\}, B = \{1, 100, 00\}$$

Solution : The solution is 1 3 1 1 3 2 2. The string is

$$\begin{aligned} A1A3A1A1A1A3A2A2 &= 100 + 1 + 100 + 100 + 1 + 0 + 0 \\ &= 1001100100100 \end{aligned}$$

$$\begin{aligned} B1B3B1B1B3B2B2 &= 1 + 00 + 1 + 1 + 00 + 100 + 100 \\ &= 1001100100100 \end{aligned}$$

Example 6.4.4 Obtain the solution for the following system of posts correspondence problem

$$A = \{ba, abb, bab\} B = \{bab, bb, abb\}.$$

Solution : Now to consider 1, 3, 2 the string babababb from set A and bababbbb from set B thus the two strings obtained are not equal. As we can try various combinations from both the sets to find the unique sequence but we could not get such a sequence. Hence there is no solution for this system.

Example 6.4.5 Does PCP with two lists $x = (b, bab^3, ba)$ and $y = (b^3, ba, a)$ have a solution ?

Solution : Now we have to find out such a sequence that strings formed by x and y are identical. Such a sequence is 2, 1, 1, 3. Hence from x and y list

$$\begin{array}{ccccccccc} 2 & 1 & 1 & 3 & & 2 & 1 & 1 & 3 \\ bab^3 & b & b & ba & = & ba & b^3 & b^3 & a \end{array}$$

which forms bab^3b^3a . Thus PCP has a solution.

Review Questions

1. What is post correspondence problem ? Explain with example.

SPPU : Dec.-10, Marks 8

2. Write short note on - Post correspondence problem.

**SPPU : May-07, 12, 14, Dec.-12, Marks 4, Dec.-11, Marks 8,
May-18, End Sem, Marks 4, Dec.-19, End Sem, Marks 4**

Part III : Complexity Classes

6.5 Time and Space Measures

- The efficiency of an algorithm can be decided by measuring the performance of an algorithm. We can measure the performance of an algorithm by computing two factors.

1. Amount of **time** required by an algorithm to execute.

2. Amount of **storage** required by an algorithm.

Hence we define two terms - Time Complexity and Space Complexity

- **Time complexity** of an algorithm means the amount of time taken by an algorithm to run. By computing time complexity we come to know whether the algorithm is slow or fast.
- **Space complexity** of an algorithm means the amount of space (memory) taken by an algorithm. By computing space complexity we can analyze whether an algorithm requires more or less space.
- To choose the best algorithm, we need to check efficiency of each algorithm. The efficiency can be measured by computing time complexity of each algorithm. Asymptotic notation is a shorthand way to represent the time complexity.
- Using asymptotic notations we can give time complexity as “fastest possible”, “slowest possible” or “average time”.
- Various notations such as Ω , Θ and O used are called **asymptotic notions**.

Big Oh Notation

The **Big oh** notation is denoted by 'O'. It is a method of representing the **upper bound** of algorithm's running time. Using big oh notation we can give longest amount of time taken by the algorithm to complete.

Definition

Let $F(n)$ and $g(n)$ be two non-negative functions.

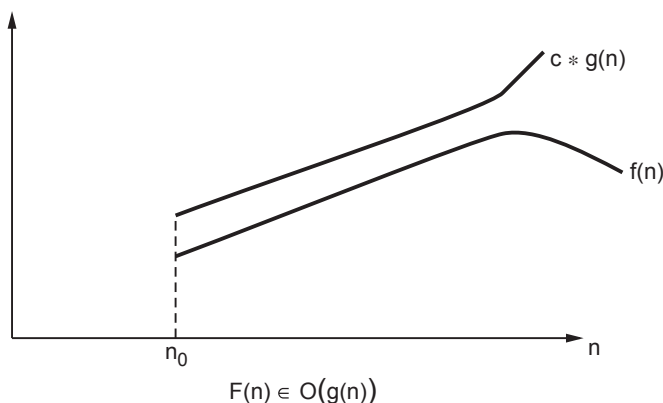


Fig. 6.5.1

Let n_0 and constant c are two integers such that n_0 denotes some value of input and $n > n_0$. Similarly c is some constant such that $c > 0$. We can write

$$f(n) \leq c * g(n)$$

then $f(n)$ is big oh of $g(n)$. It is also denoted as $f(n) \in O(g(n))$. In other words $f(n)$ is less than $g(n)$ if $g(n)$ is multiple of some constant c .

Example 6.5.1 Consider function $f(n) = 2n + 2$ and $g(n) = n^2$. Then we have to find some constant c , so that $f(n) \leq c * g(n)$. As $F(n) = 2n + 2$ and $g(n) = n^2$ then we find c for $n = 1$ then

Solution :

$$f(n) = 2n + 2 = 2(1) + 2$$

$$f(n) = 4$$

and $g(n) = n^2 = (1)^2$

$$g(n) = 1$$

i.e. $f(n) > g(n)$

If $n = 2$ then,

$$f(n) = 2(2) + 2 = 6$$

$$g(n) = (2)^2$$

$$g(n) = 4$$

i.e. $f(n) > g(n)$

If $n = 3$ then,

$$f(n) = 2(3) + 2 = 8$$

$$g(n) = (3)^2$$

$$g(n) = 9$$

i.e. $f(n) < g(n)$ is true.

Hence we can conclude that for $n > 2$, we obtain

$$f(n) < g(n)$$

Thus always upper bound of existing time is obtained by big oh notation.

6.5.1 Order of Growth

Measuring the performance of an algorithm in relation with the input size n is called **order of growth**. For example, the order of growth for varying input size of n is as given below.

n	$\log n$	$n \log n$	n^2	2^n
1	0	0	1	2
2	1	2	4	4
4	2	8	16	16
8	3	24	64	256
16	4	64	256	65,536
32	5	160	1024	4,294,967,296

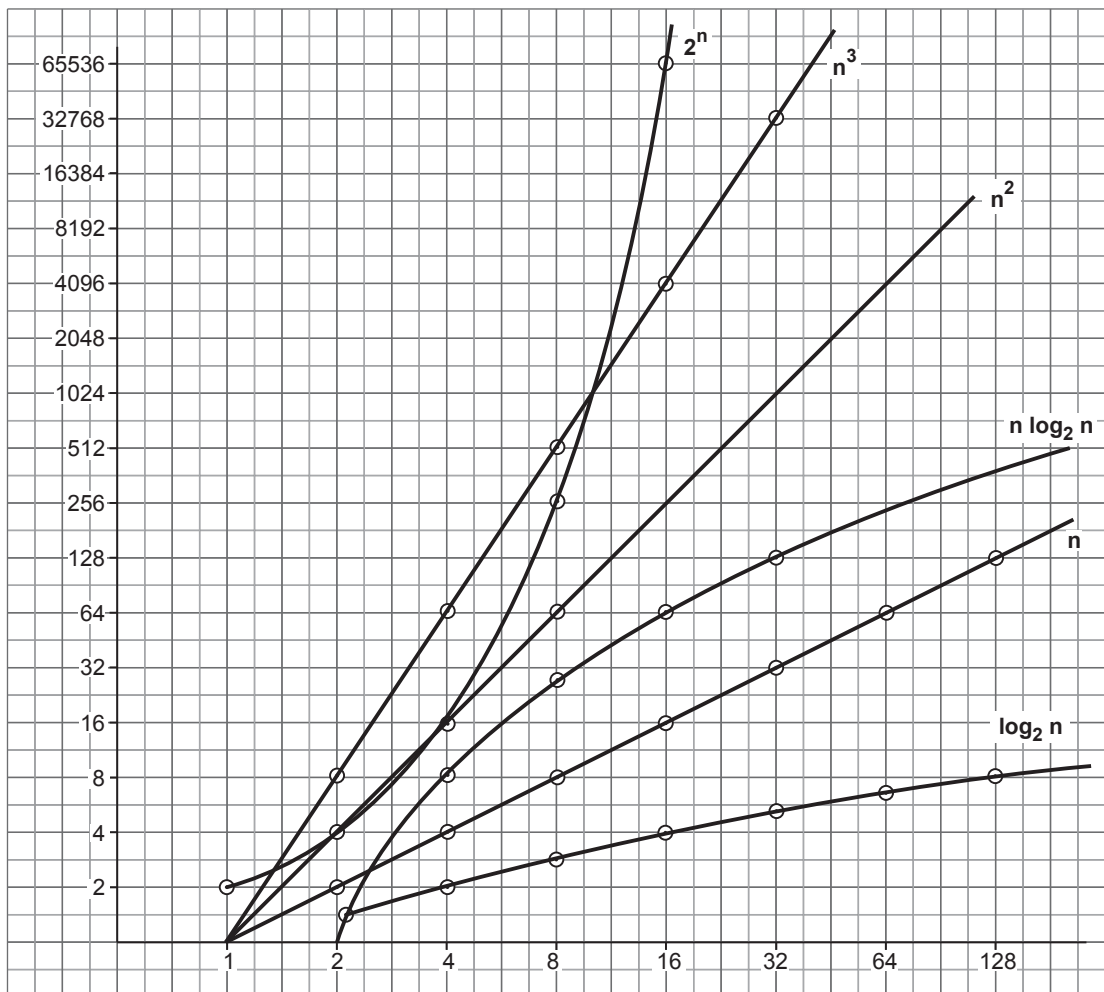


Fig. 6.5.2 Rate of growth of common computing time function

From the above drawn graph it is clear that the logarithmic function is the slowest growing function. And the exponential function 2^n is fastest and grows rapidly with varying input size n . The exponential function gives huge values even for small input n . For instance : For the value of $n = 16$ we get $2^{16} = 65536$.

6.6 The Class P and NP

SPPU : Dec.-15,18,19, Marks 4

There are two groups in which a problem can be classified. The first group consists of the problems that can be solved in polynomial time. For example : searching of an element from the list $O(\log n)$, sorting of elements $O(\log n)$.

The second group consists of problems that can be solved in non-deterministic polynomial time. For example : Knapsack problem $O(2^{n/2})$ and Travelling Salesperson problem ($O(n^2 2^n)$).

- Any problem for which answer is either yes or no is called decision problem. The algorithm for decision problem is called **decision algorithm**.
- Any problem that involves the identification of optimal cost (minimum or maximum) is called optimization problem. The algorithm for optimization problem is called **optimization algorithm**.
- **Definition of P** - Problems that can be solved in polynomial time. ("P" stands for polynomial). The polynomial time is nothing but the time expressed in terms of polynomial.
Examples - Searching of key element, Sorting of elements, All pair shortest path.
- **Definition of NP** - It stands for "non-deterministic polynomial time". Note that NP does not stand for "non-polynomial".
- Examples - Travelling Salesperson problem, Graph coloring problem, Knapsack problem, Hamiltonian circuit problems.
- The NP class problems can be further categorized into NP-complete and NP-hard problems.

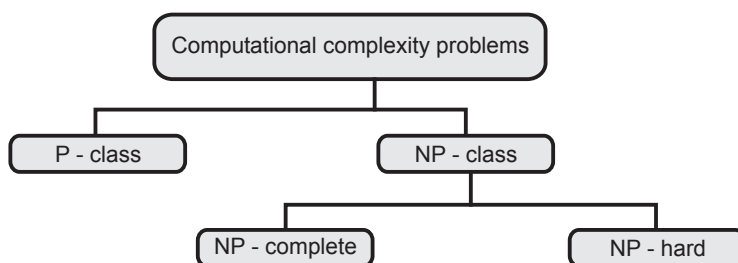


Fig. 6.6.1

- A problem D is called NP-complete if -
 - i) It belongs to class NP
 - ii) Every problem in NP can also be solved in polynomial time.
- If an NP-hard problem can be solved in polynomial time then all NP-complete problems can also be solved in polynomial time.
- All NP-complete problems are NP-hard but all NP-hard problems cannot be NP-complete.
- The NP class problems are the decision problems that can be solved by non-deterministic polynomial algorithms.
- **Computational complexity** - The computational problems is an infinite collection of instances with a solution for every instance.

In computational complexity the solution to the problem can be "yes" or "no" type. Such type of problems are called **decision problem**. The computational problems can be **function problem**. The function problem is a computational problem where single output is expected for every input. The output of such type of problems is more complex than the decision problem.

The computational problems also consists of a class of problems, whose output can be obtained in polynomial time.

- **Complexity classes** - The complexity classes is a set of problems of related complexity. It includes function problems, P classes, NP classes, optimization problem.
- **Intractability** - Problems that can be solved by taking a long time for their solutions is known as intractable problems.

If NP is not same as P then NP complete problems are called intractable problems.

6.6.1 Difference between P Class and NP Class Problems

P Class	NP Class Problem
Problems that can be solved in polynomial time are called P-class problems.	Problems that can be solved in non-deterministically Polynomial time are called NP class problem.
These are simple to solve.	These are complex to solve.
Examples : Searching an element from unordered list, sorting the elements.	Examples : Traveling salesperson problem, knapsack problem.

Review Questions

1. Explain with example - computational complexity.

SPPU : Dec.-15, End Sem, Marks 2

2. Differentiate between P class and NP class problems.

SPPU : Dec.-15, End Sem, Dec.-18,19, End Sem, Marks 4

6.7 Examples of Problems in P

SPPU : Dec.-17,18, Marks 8

Kruskal's algorithm : In Kruskal's algorithm the minimum weight is obtained. In this algorithm also the circuit should not be formed. Each time the edge of minimum weight has to be selected, from the graph. It is not necessary in this algorithm to have edges of minimum weights to be adjacent. Let us solve one example by Kruskal's algorithm.

Example : Find the minimum spanning tree for the following figure using Kruskal's algorithm.

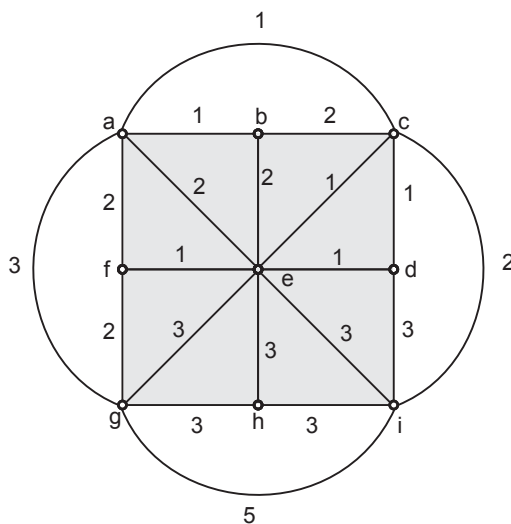


Fig. 6.7.1

In Kruskal's algorithm, we will start with some vertex and will cover all the vertices with minimum weight. The vertices need not be adjacent.

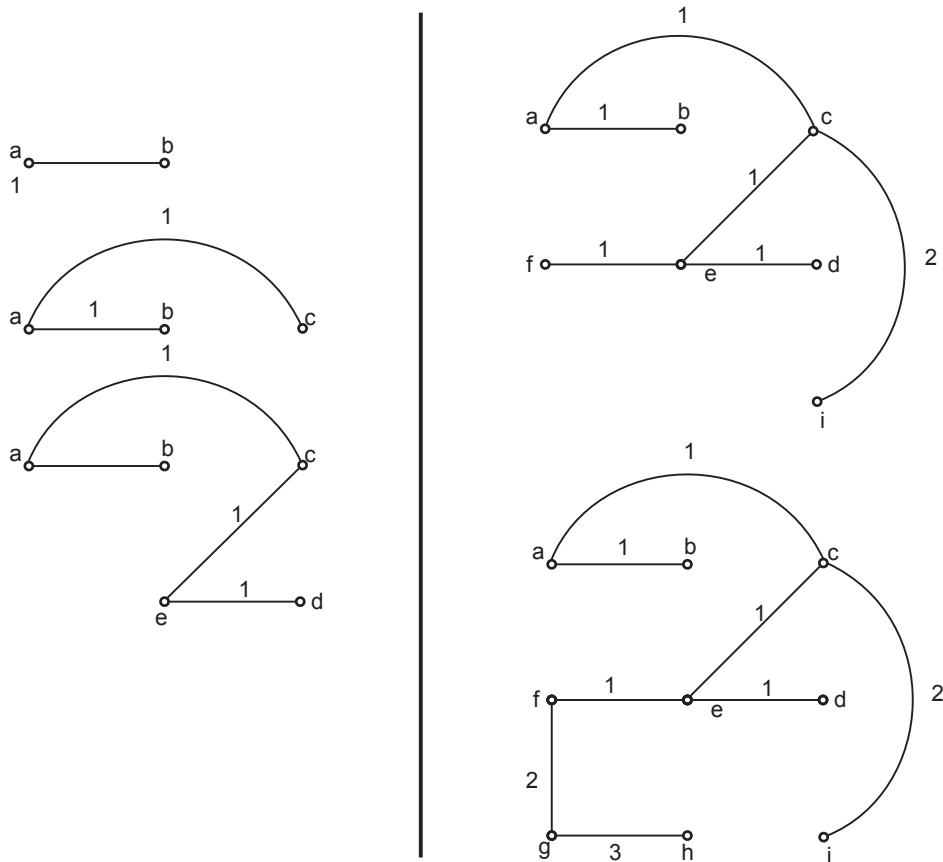


Fig. 6.7.2

Review Question

1. What is Kruskal's algorithm ? How can we solve this problem using Turing machine ?

SPPU : Dec.-17, 18, End Sem, Marks 8

6.8 Examples of Problems in NP

SPPU : May-16,18, Dec.-17,19, Marks 8

Travelling Salesman's Problem (TSP) : This problem can be stated as " Given a set of cities and cost to travel between each pair of cities, determine whether there is a path that visits every city once and returns to the first city. Such that the cost travelled is less".

For example :

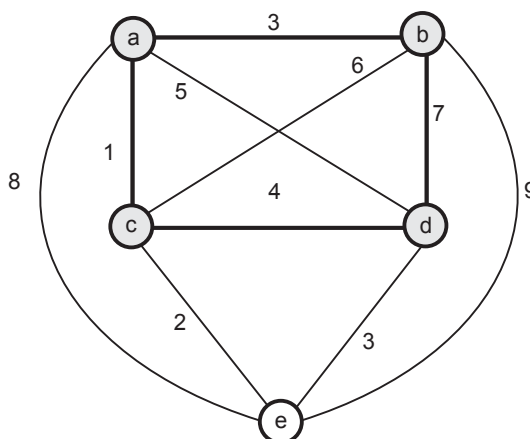


Fig. 6.8.1

The tour path will be **a-b-d-e-c-a** and total cost of tour will be **16**.

This problem is NP problem as there may exist some path with shortest distance between the cities. If you get the solution by applying certain algorithm then Travelling Salesman problem is **NPComplete Problem**. If we get no solution at all by applying an algorithm then the travelling salesman problem belongs to **NP hard class**.

Review Question

1. What do you mean by NP problems ? Justify why the Traveling Salesman problem is a NP problem ?

SPPU : May-16, End Sem, May-18, End Sem, Dec.-17, 19, Marks 8

6.9 NP-completeness and NP-hard Problems

SPPU : Dec.-16, Marks 8

- As we know, P denotes the class of all deterministic polynomial language problems and NP denotes the class of all non-deterministic polynomial language problems. Hence

$$P \subseteq NP.$$

- The question of whether or not

$$P = NP$$

holds, is the most famous outstanding problem in the computer science.

- Problems which are known to lie in P are often called as *tractable*. Problems which lie outside of P are often termed as *intractable*. Thus, the question of whether $P = NP$ or $P \neq NP$ is the same as that of asking whether there exist problems in NP which are intractable or not.

The relationship between P and NP is depicted by Fig. 6.9.1.

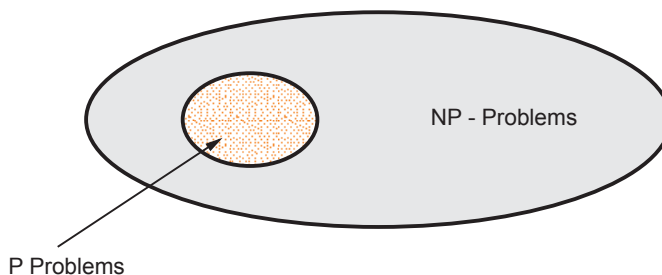


Fig. 6.9.1 P and NP problems assuming $P \neq NP$

- We don't know if $P = NP$. However in 1971, S. A. Cook proved that a particular NP problem known as SAT (Satisfiability of sets of Boolean clauses) has the property that, if it is solvable in polynomial time, so are all NP problems. This is called a **"NP-complete"** problem.
- Let L_1 and L_2 are two problems then problem L_1 reduces to L_2 if and only if there is a way to solve L_1 by deterministic polynomial time algorithm using a deterministic algorithm that solves L_2 in polynomial time.
 L_1 reduces to L_2 can be denoted as $L_1 \propto L_2$. In other words we can say that if there exists any polynomial time algorithm which solves L_2 then we can solve L_1 in polynomial time. We can also state that if $L_1 \propto L_2$ and $L_2 \propto L_3$ then $L_1 \propto L_3$.
- A NP problem such that, if it is in P, then $NP = P$. If a (not necessarily NP) problem has this same property then it is called **"NP-hard"**. Thus the class of NP-complete problem is the intersection of the NP and NP-hard classes.
- Normally the decision problems are NP-complete but optimization problems are NP-hard. However if problem L_1 is a decision problem and L_2 is optimization problem then it is possible that $L_1 \propto L_2$. For instance the Knapsack decision problem can be Knapsack optimization problem.

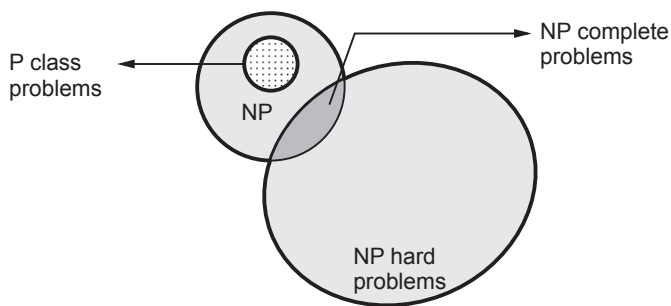


Fig. 6.9.2 Relationship between P, NP, NP-complete and NP-hard problems

- There are some NP-hard problems that are not NP-complete. For example *halting problem*. The halting problem states that : “Is it possible to determine whether an algorithm will ever halt or enter in a loop on certain input ? ”

Two problems P and Q are said to be polynomially equivalent if and only if $P \propto Q$ and $Q \propto P$.

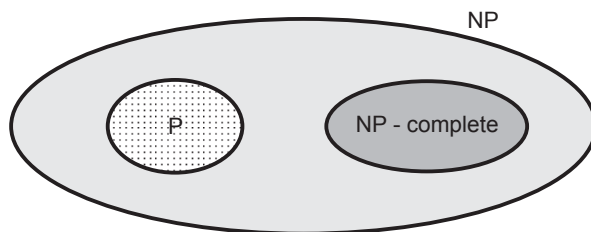


Fig. 6.9.3

Review Question

1. What do you mean by NP complete problem ? List all the problems in this class and explain any one with suitable example.

SPPU : Dec.-16, End Sem, Marks 8

6.10 Polynomial Time Reduction

SPPU : May-08,16, Dec.-18,19, Marks 8

To prove whether particular problem is NP complete or not we use polynomial time reducibility. That means if

$$A \xrightarrow{\text{Poly}} B \text{ and } B \xrightarrow{\text{Poly}} C \text{ then } A \xrightarrow{\text{Poly}} C.$$

The **reduction** is an important task in NP completeness proofs. This can be illustrated by Fig. 6.10.1. (See Fig. 6.10.1 on next page.)

Various types of reductions are :

Local replacement - In this reduction $A \rightarrow B$ by dividing input to A in the form of components and then these components can be converted to components of B.

Component design - In this reduction $A \rightarrow B$ by building special component for input B that enforce properties required by A.

The reduction can be denoted as $A \leq_T^P B$.

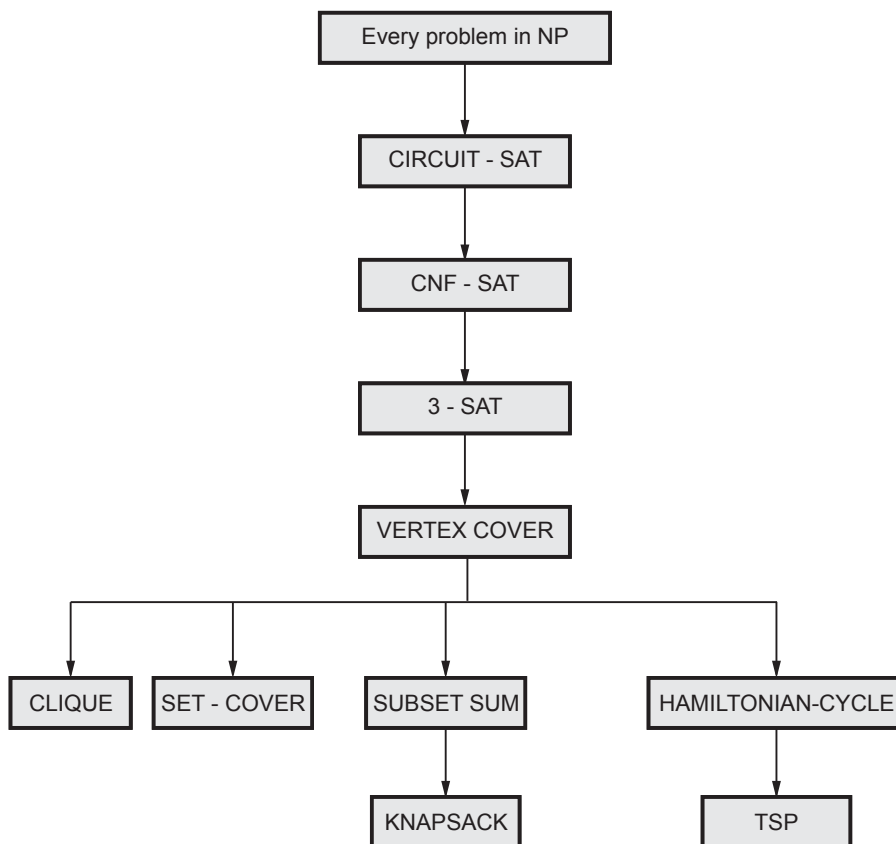


Fig. 6.10.1 Reduction in NP completeness

6.10.1 Proving NP Problems

1. HAMILTONIAN CYCLE : This is a problem in which graph G is accepted as input and it is asked to find a simple cycle in G that visits each vertex of G exactly once and returns to its starting vertex. Such a cycle is called an **Hamiltonian cycle**.

Theorem : HAMILTONIAN CYCLE is in NP.

Proof : Let A be some non-deterministic algorithm to which graph G is given as input. The vertices of graph are numbered from 1 to N . We have to call the algorithm recursively in order to get the sequence S . This sequence will have all the vertices without getting repeated. The vertex from which the sequence starts must be ended at the end. This check on the sequence S must be made in polynomial time n . Now if there is a Hamiltonian cycle in the graph then algorithm will output "yes". Similarly if we get output of algorithm as "yes" then we could guess the cycle in G with every vertex appearing exactly once and the first visited vertex getting visited at the last. That means

A non-deterministically accepts the language HAMILTONIAN CYCLE. It is therefore proved that HAMILTONIAN CYCLE is in NP.

2. CIRCUIT-SAT : This is a problem in which a Boolean circuit is taken as input with single output node. And then finds whether there is an assignment of values to the circuit's input so that we get its output value as "1". This assignment of values is called **satisfying assignment**.

Theorem : CIRCUIT-SAT is in NP.

Proof :

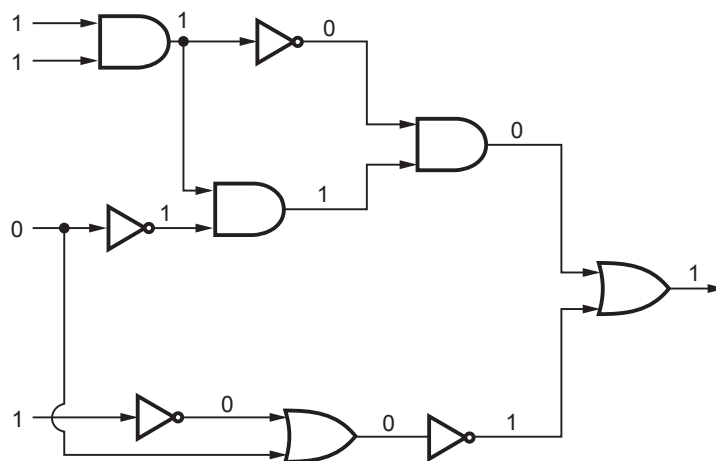


Fig. 6.10.2

The logic gates that are used are

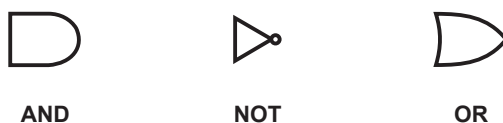


Fig. 6.10.3

Let us construct a non-deterministic algorithm. Which works in polynomial time. Then we should have **choose ()** method which can guess the values of input node as well as the output, then the algorithm says "no" if we get output of Boolean circuit as 0. Similarly the algorithm says "yes" if we get output of Boolean circuit as 1.

Now from the output of algorithm we can guess the inputs to logic gates in the circuit. If the algorithm has output "yes" then we can say that Boolean circuit has satisfying input values. Thus we can say that CIRCUIT-SAT is in NP.

Theorem : Prove that a directed Hamiltonian cycle $\propto$ the travelling salesman decision problem.

SPPU : May-08, Marks 6

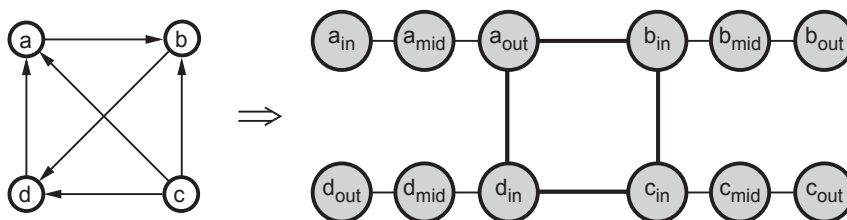
Proof

The directed Hamiltonian cycle is a cycle in which each vertex of a directed graph is visited only once. To prove that directed Hamiltonian cycle (DHC) $\propto$ Travelling Salesperson Problem (TSP) we will apply following steps -

If there are two problems A and B and if we wish to prove that B is NP complete then -

- 1) Pick up a known NP complete problem A.
- 2) Reduce A to B. This can be done in following steps.
 - a) Describe a transformation that maps the instance of A to instance of B in such a manner that "yes" for B = "yes" for A.
 - b) Prove that this transformation runs in polynomial time.
- 3) This proves that problem B is also NP complete.

To prove that DHC $\propto$ TSP we will reduce Directed Hamiltonian cycle to undirected Hamiltonian Cycle. The conversion is as follows -



In this undirected graph if there exists a Hamiltonian cycle, then that denotes visiting cities in the order of nodes being visited in Hamiltonian cycle.

If no Hamiltonian cycle exists that means TSP has no solution. The Hamiltonian cycle problem is NP complete. This proves that TSP is also NP complete.

Review Question

1. What do you mean by polynomial time reduction ? Explain with suitable example.

SPPU : May-16, End Sem, Marks 8, Dec.-18, 19, End Sem, Marks 6

6.11 Satisfiability Problems

SPPU : Dec.-16,18, Marks 8

6.11.1 The Satisfiability Problems

The boolean expression are created using -

- 1) The variables whose value can be 0 or 1.
- 2) The operators that are used in expression can be $\vee$, $\wedge$. The $\vee$ means OR and $\wedge$ means AND, operators.
- 3) The unary operator $\neg$ stands for negation.
- 4) The parenthesis are used to group the operands and operators in the expression. The highest precedence is to $\neg$, then $\wedge$, then $\vee$.

For example :

The boolean expression is $(x \wedge y) \vee (y \wedge z)$. If $y \wedge z$ and $x \wedge y$ both are true then the boolean expression is true if $(x \wedge y)$ and $(y \wedge z)$ both are false the boolean expression is false.

- The study of boolean functions generally concerned with set of **truth assignments** (i.e. assignments of 0 or 1 to each variable). Due to this assignment the function becomes true.
- SAT problem : Does there exist a truth assignment making function true ?
- $(x \wedge y \wedge z)$ is satisfiable when $x = 1$, $y = 1$ and $z = 1$.

But $x \wedge (\neg y)$ is not satisfiable.

6.11.2 Representing SAT Instances

- An instance of SAT is a boolean function.
- The symbols $\wedge$, $\vee$, $\neg$, (and) are used in these instances.
- The variable x_i is represented by symbol x followed by 0's and 1's. For example x_2 is represented as x_{10} . (Here 2 is replaced by binary value 10).
- **Example :** $(x_1 \vee x_2) \wedge (x_2 \vee \neg x_3)$ is represented as $(x_1 \vee x_{10}) \wedge (x_{10} \vee \neg x_{11})$

6.11.3 NP Completeness of SAT Problem

In 1971, **Stephen Cook** proved one popular theorem which is popularly known as Cook's theorem. This theorem refers a specific problem as is $P = NP$? That means if we find a polynomial time algorithm for **satisfiability** then P is definitely equal to NP and then we can solve all the NP problems in polynomial time, no matter how different they might be from the **satisfiability**. He states the theorem in following manner.

Theorem : The satisfiability problem is NP complete.

Proof :

- **Deterministic algorithms** are those algorithms which **accept** or **reject** the strings. **Non-deterministic algorithms** are those algorithms which generally **accept** the strings but generally **do not reject** strings.
- A problem X is an element of NP if there is a non-deterministic polynomial time algorithm that solves X .
- All the algorithms can be represented by Turing Machine $TM - M$.
- Hence, when we say $X \in NP$, then that means there exists a non-deterministic turing machine M_{NP} which accepts any element of problem X in polynomial time.
- By Cook's theorem we have to show that there exists some boolean expression which is satisfiable (i.e. the expression evaluates to TRUE). Note that the Boolean expression must be in **Conjunctive Normal Form** (CNF). The Boolean expression must be created for the element $x \in X$.
- Let, M_{NP} will be a turing machine and a polynomial time bound can be denoted by $p(n)$.
- The turing machine will have a state diagram and input tape. The tape consists of cells arranged adjacently. It will be a two way tape which is infinitely long in both the directions.
- Σ is the set of input symbols that can be placed on the input tape where each cell consists of single letter.
- Γ is set symbols which is used by TM while processing the input. The empty cells are occupied by blank symbols (denoted by Δ).
- Q is a set of states. Along with other states Q consists of three type of states.

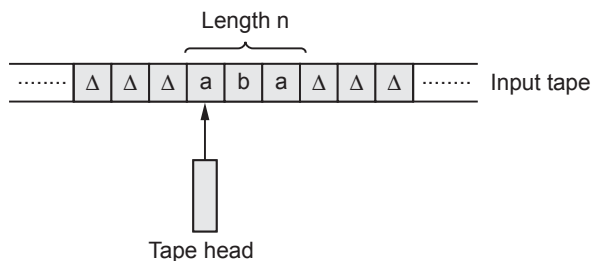
q_0 - Initial state

q_{ACCEPT} - It is accept state which represents that input string is accepted.

q_{REJECT} - It is a reject state which represents that input string is not accepted.

When the turing machine M_{NP} is in state q_{ACCEPT} or q_{REJECT} then it gets halted and there is no further transition from this state.

- The δ is a transition function which determines the move of the Turing Machine M_{NP} .
There will be two possible directions for each move L or R.



- Each element will be accessed within **unit of time**, and then the transition is made. To accept x the M_{NP} must enter in q_{ACCEPT} state within $p(n)$ polynomial time but it is not necessary for M_{NP} to enter in q_{REJECT} state to reject x .
- Each string fed to machine M_{NP} must be **modeled in CNF form**. Hence it is necessary to define two things - 1) Number of **variables** representing conditions in M_{NP} and 2) **Clauses** to represent legal accepting string for M_{NP} . Let us get introduced with the variables.
- **$Q[i, q_m]$** where i runs from 0 to $p(n)$ and q_m runs through all states of M_{NP} . The $Q[i, q_m]$ represents that at time i , M_{NP} is in state q_m . **$H[i, j]$** where i runs from 0 to $p(n)$ and j runs from $-p(n)$ to $p(n) + 1$. This variable $H[i, j]$ represents that at time i , the Turing machine M_{NP} will scan the cell j on the input tape. This will be performed in $p(n)$ transitions. **$S[i, j, s_m]$** where i runs from 0 to $p(n)$, j runs from $-p(n)$ to $p(n) + 1$ and s_m runs through all the tape symbols. This variable $S[i, j, s_m]$ represents the content s_m of j^{th} cell at time i .
- There will be six clauses which can make accepting sequence of M_{NP} . These are -
 - C - This clause represents that at each time i , M_{NP} will be in one and only one **state**.
 - D - This clause represents that at each time i , M_{NP} will be **scanning** one and only one cell of the tape.
 - E - This clause represents that at each time i there will be only one **symbol** in each cell, of the input tape.
 - F - This clause states that machine (i.e. TM) will be in q_0 state initially with properly **positioned read-write head** for input element x .
 - G - This clause represents that by time $p(n)$, M_{NP} will enter in **q_{ACCEPT} state**.
 - H - This clause represents that all the **transitions** can be correctly applied.

Now we will represent each clause as a set **for clause C** -

- i) Add a clause $\{Q[i, q_1], Q[i, q_2], \dots, Q[i, q_k]\}$ where i is the time and $q_1, q_2, \dots, q_k$ represents number of states $\in Q$.
- ii) Add a clause $\{\overline{Q[i, q_r]}, \overline{Q[i, q_t]}\}$ where q_r and q_t represent paired states.

For each unit of time i .

The i) Case represents that M_{NP} will be at least in one state at any time i .

The ii) Case represents that the M_{NP} will not be in more than one state at any time i . The time i will be from 0 to $p(n)$.

- **For clause D** -

- i) Add a clause $\{H[i, -p(n)], H[i, -p(n)+1], \dots, H[i, p(n)+1]\}$
- ii) Add a clause $\{\overline{H[i, r]}, \overline{H[i, t]}\}$ where r and t represents pair of tape cells.

The i) Case represents that at every time i M_{NP} will be scanning at least one cell. The ii) Case represents that at every time i M_{NP} will not scan more than one cell.

- **For clause E** -

Add a clause that any time i each cell of tape contains at least one symbol and add a clause that guarantees that at any time i no cell contains more than one tape symbol.

- **For clause F** -

- i) Add a clause $Q[0, q_0]$ i.e. M_{NP} is in start state.
- ii) Add a clause $H[0, 1]$ i.e. The tape head is positioned at cell 1.
- iii) Add a clause such that input string is on tape at time 0.
- iv) Add a set of clauses that guarantees that at time 0 rest of the tape cells contain blank characters.

- **For clause G** -

- i) Add a clause $\{Q[p(n), q_{ACCEPT}]\}$ i.e. M_{NP} has entered in Accept state at any polynomial time $p(n)$. This causes M_{NP} to accept the input string and there is no further transition from q_{ACCEPT} .

- **For clause H** -

Add a clause for mapping function δ where $(q_{curr}, s_{curr}, q_{next}, s_{next}, dir)$ be the element of δ .

This statement will be modeled in CNF form. That means if at time i , M_{NP} is in q_{curr} and s_{curr} symbol is read then move to q_{next} state and tape may contain s_{next} symbol. This move will be based on **dir** which can be L or R.

- The final expression will be

$$\text{EXP} = C \cup D \cup E \cup F \cup G \cup H$$

EXP is satisfiable if and only if M_{NP} accepts $x = x_1 x_2 x_3 \dots x_n$. This expression can be constructed from turing machine M_{NP} in polynomial time. This proves the theorem.

Review Questions

1. What is SAT problem ? Explain in detail.

SPPU : Dec.-16, End Sem, Marks 8

2. Explain the satisfiability problem with an example.

SPPU : Dec.-18, End Sem, Marks 4

3. Prove that the satisfiability problem is NP-complete.

SPPU : Dec.-18, End Sem, Marks 6

6.12 Restricted Satisfiability Problem

The intermediate form of SAT problem is **3-SAT** problem. It is much easier to use 3-SAT instead of SAT. 3-SAT is still a problem of satisfiability of boolean expression.

Definition : A 3SAT problem is a problem which takes a Boolean formula S in CNF form with each clause having **exactly three literals** and check whether S is satisfied or not. [Note that CNF means each literal is ORed to form a clause, and each clause is ANDed to form Boolean formula S].

Following formula is an instance of 3-SAT problem :

$$(\bar{a} + b + \bar{g}) (c + \bar{e} + f) (\bar{b} + d + \bar{f}) (a + e + \bar{h})$$

6.12.1 Normal Forms for Boolean Expression

A Boolean formula is in **Conjunctive Normal Form (CNF)** if it is formed as collection of subexpressions. These subexpressions are called **clauses**.

For example

$$(\bar{a} + b + d + \bar{g}) (c + \bar{e}) (\bar{b} + d + \bar{f} + h) (a + c + e + \bar{h})$$

This formula evaluates to 1 if b, c, d are 1.

The CNF-SAT is a problem which takes Boolean formula in CNF form and checks whether any assignment is there to Boolean values so that formula evaluates to 1.

6.12.2 Converting Expressions to CNF

- There are two parts of this conversion -

- 1) Push all $\neg$'s down the expression so that only negations are of variables, the boolean expression becomes AND and OR of literals.

2) Write Boolean expression in CNF form, we will get new variables.

- **DeMorgan's Law**

$$1) \neg (A \wedge B) = \neg (A) \vee \neg (B)$$

Pushing $\neg$ below and converting $\wedge$ to $\vee$.

$$2) \neg (A \vee B) = \neg (A) \wedge \neg (B)$$

Pushing $\neg$ below and converting $\vee$ to $\wedge$.

$$3) \neg (\neg A) = A \text{ i.e. double negation.}$$

For example

$$\begin{aligned} \neg ((\neg x + y) (x + y)) \\ &= (\neg (\neg x + y) + \neg (x + y)) \\ &= x \neg y + \neg (x + y) \\ &= x \neg y + (\neg x + \neg y) \\ &= x \neg y \neg (x + y) \end{aligned}$$

Theorem : CNF-SAT is in NP complete.

Proof : Let S be the Boolean formula for which we can construct a simple non-deterministic algorithm which can guess the values of variables in Boolean formula and then evaluates each clause of S. If all the clauses evaluate S to 1 then S is satisfied. Thus CNF-SAT is in NP-complete.

Theorem : 3SAT is in NP complete.

Proof : Let S be the Boolean formula having 3 literals in each clause for which we can construct a simple non-deterministic algorithm which can guess an assignment of Boolean values to S. If the S is evaluated as 1 then S is satisfied. Thus we can prove that 3SAT is in NP-complete.

□□□

Notes

[illegible]